

CPE 342 Machine Learning

Assignment 2: Training Models via Iterative Approach

Gradient Descent — Quadratic Model Fitting

67070501042 วิศิษฐ์ สุวรรณนาวี

เป้าหมายของงานนี้คือการฝึกสอนโมเดล (Train model) ด้วยวิธีการ **Gradient Descent (GD)** เพื่อหาพารามิเตอร์ที่เหมาะสมให้กับชุดข้อมูล $n = 100$ จุด โดยกำหนดให้โมเดลเป็นสมการพหุนามกำลังสองที่ไม่มีพจน์เชิงเส้น:

$$\hat{y} = C_0 + C_1x^2$$

1. ฟังก์ชันความสูญเสีย (Loss Function)

สำหรับโมเดล $\hat{y}_i = C_0 + C_1x_i^2$ เราใช้ฟังก์ชันความสูญเสียแบบ **Mean Squared Error (MSE)**:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

เมื่อแทนค่า $\hat{y}_i$ ลงไป จะได้:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - C_0 - C_1x_i^2)^2$$

2. การหาอนุพันธ์ย่อย (Gradient of Each Coefficient)

เพื่อใช้ Gradient Descent เราต้องหาอนุพันธ์ย่อย (Partial derivatives) ของ L เทียบกับ C_0 และ C_1

ให้ $e_i = y_i - C_0 - C_1x_i^2$

Gradient เทียบกับ C_0 :

$$\frac{\partial L}{\partial C_0} = \frac{1}{n} \sum_{i=1}^n 2e_i \cdot \frac{\partial e_i}{\partial C_0} = \frac{1}{n} \sum_{i=1}^n 2e_i \cdot (-1)$$

$$\frac{\partial L}{\partial C_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1x_i^2)$$

Gradient เทียบกับ C_1 :

$$\frac{\partial L}{\partial C_1} = \frac{1}{n} \sum_{i=1}^n 2e_i \cdot \frac{\partial e_i}{\partial C_1} = \frac{1}{n} \sum_{i=1}^n 2e_i \cdot (-x_i^2)$$

$$\frac{\partial L}{\partial C_1} = -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2) x_i^2$$

3. กฎการอัปเดตพารามิเตอร์ (Update Rules)

ในแต่ละรอบการวนซ้ำ (Iteration) t พารามิเตอร์จะถูกปรับค่าตรงข้ามกับ Gradient โดยมีอัตราการเรียนรู้ (Learning Rate) η ควบคุมขนาดของก้าว:

$$C_0^{(t+1)} = C_0^{(t)} - \eta \cdot \frac{\partial L}{\partial C_0}$$

$$C_1^{(t+1)} = C_1^{(t)} - \eta \cdot \frac{\partial L}{\partial C_1}$$

4. สรุปผลและโค้ดการทำงานจริง

เมื่อเขียนโค้ดด้วย NumPy พื้นฐาน และกำหนด hyperparameters คือ $\eta = 0.01$ จำนวน 5,000 รอบ พบว่าค่า Loss สามารถลู่เข้าหาจุดต่ำสุดได้อย่างสมบูรณ์ และพารามิเตอร์ C_0, C_1 ก็มีความเสถียร (รายละเอียดกราฟ ผลลัพธ์ และโค้ดอยู่ใน Appendix ด้านล่าง)

Appendix: Full Jupyter Notebook Code & Output

รายละเอียดต่อไปนี้เป็นโค้ด Python ที่ใช้แก้ปัญหาข้างต้น พร้อมผลลัพธ์และกราฟ (พิมพ์ด้วยฟอนต์ TH Sarabun New) ซึ่งได้รับจริงจาก Jupyter Notebook

Jupyter Notebook: Assignment_2_GD.ipynb

โค้ด Python และผลลัพธ์ทั้งหมดจาก Jupyter Notebook (รันสมบูรณ์)

2. Library & Font Setup

Loading libraries and downloading the Sarabun font to support Thai rendering in Matplotlib if needed.

In [1]:

```
1 # ดาวน์โหลดฟอนต์ TH Sarabun New สำหรับใช้ใน Matplotlib รันบน( Colab/
   # Jupyter ได้)
2 !wget -q https://github.com/Phonbopit/sarabun-webfont/raw/master/
   fonts/thсарunnew-webfont.ttf
3
4 import numpy as np
5 import pandas as pd
6 import matplotlib.pyplot as plt
7 import matplotlib as mpl
8 import matplotlib.font_manager as fm
9 import urllib.request
10 import os
11
12 # การตั้งค่าฟอนต์
13 font_path = 'thсарunnew-webfont.ttf'
14 if not os.path.exists(font_path):
15     urllib.request.urlretrieve('https://github.com/Phonbopit/
   sarabun-webfont/raw/master/fonts/thсарunnew-webfont.ttf',
   font_path)
16 fm.fontManager.addfont(font_path)
17 mpl.rc('font', family='TH Sarabun New', size=14)
18 mpl.rcParams['axes.unicode_minus'] = False #
   ป้องกันปัญหาเครื่องหมายลบแสดงเป็นสี่เหลี่ยม
19
20 print("Library and font setup complete")
```

Out[1]:

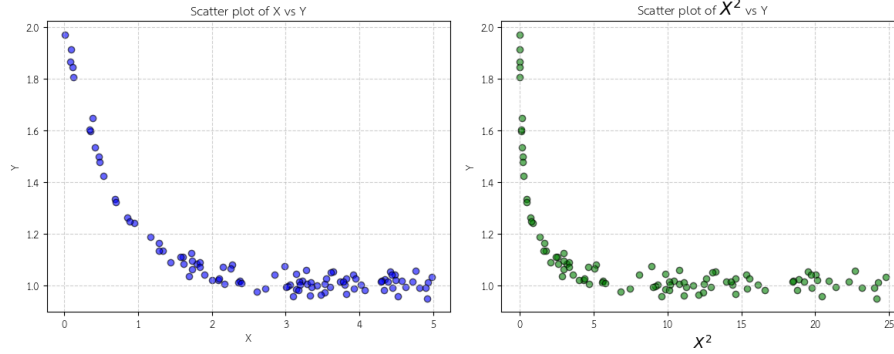
'wget' is not recognized as an internal or external command,
operable program or batch file.
Library and font setup complete

3. Data Loading & Exploratory Data Analysis (EDA)

Read the data from the CSV file and plot scatter plots to observe the relationship between x and y .

In [2]:

```
1 # อ่านข้อมูล
2 df = pd.read_csv('CPE342_Assignment 2_Data.csv')
3 X = df['X'].values
4 Y = df['Y'].values
5
6 # แสดงข้อมูลเบื้องต้น
7 display(df.head())
8
9 # พล็อตกราฟเพื่อดูแนวโน้ม
10 plt.figure(figsize=(12, 5))
11
12 # กราฟ X กับ Y
13 plt.subplot(1, 2, 1)
14 plt.scatter(X, Y, color='blue', alpha=0.6, edgecolor='k')
15 plt.title('Scatter plot of X vs Y')
16 plt.xlabel('X')
17 plt.ylabel('Y')
18 plt.grid(True, linestyle='--', alpha=0.6)
19
20 # กราฟ  $X^2$  กับ Y
21 plt.subplot(1, 2, 2)
22 plt.scatter(X**2, Y, color='green', alpha=0.6, edgecolor='k')
23 plt.title('Scatter plot of  $X^2$  vs Y')
24 plt.xlabel(' $X^2$ ')
25 plt.ylabel('Y')
26 plt.grid(True, linestyle='--', alpha=0.6)
27
28 plt.tight_layout()
29 plt.show()
```



Task 1 — Loss Function

Given our model $\hat{y}_i = C_0 + C_1 x_i^2$, The loss function we use is the **Mean Squared Error (MSE)**, which computes the average squared differences (residuals) across all data points:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Substituting $\hat{y}_i$ gives our target equation:

$$L(C_0, C_1) = \frac{1}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2)^2$$

Task 2 — Gradient of Each Coefficient

To use Gradient Descent, we need to compute the partial derivatives of the loss function L with respect to each parameter. Let $e_i = y_i - C_0 - C_1 x_i^2$ be the residual error. **1.**

Gradient with respect to C_0 :

$$\frac{\partial L}{\partial C_0} = \frac{1}{n} \sum_{i=1}^n 2(y_i - C_0 - C_1 x_i^2) \cdot \frac{\partial}{\partial C_0}(-C_0)$$

$$\frac{\partial L}{\partial C_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2)$$

2. Gradient with respect to C_1 :

$$\frac{\partial L}{\partial C_1} = \frac{1}{n} \sum_{i=1}^n 2(y_i - C_0 - C_1 x_i^2) \cdot \frac{\partial}{\partial C_1}(-C_1 x_i^2)$$

$$\frac{\partial L}{\partial C_1} = -\frac{2}{n} \sum_{i=1}^n (y_i - C_0 - C_1 x_i^2) x_i^2$$

Task 3 — Update Rules

In each iteration t , the parameters are updated in the opposite direction of the gradient to minimize the loss, scaled by the learning rate η :

$$C_0^{(t+1)} = C_0^{(t)} - \eta \cdot \frac{\partial L}{\partial C_0}$$

$$C_1^{(t+1)} = C_1^{(t)} - \eta \cdot \frac{\partial L}{\partial C_1}$$

Task 4 — Gradient Descent Implementation

In this section, we implement the code to find C_0 and C_1 using basic NumPy, based on the mathematical equations derived in the previous tasks.

In [3]:

```
1 # Hyperparameters
2 learning_rate = 0.01
3 n_iterations = 5000
4 n = len(Y)
5
6 # Initialize parameters
7 C0 = 0.0
8 C1 = 0.0
9
10 # Prepare lists to store history (for visualization later)
11 history_loss = []
12 history_C0 = []
13 history_C1 = []
14
15 # Pre-compute X^2 to reduce computation inside the loop
16 X_sq = X**2
17
18 # Gradient Descent Loop
19 for i in range(n_iterations):
20     # 1. Calculate Predictions
21     Y_pred = C0 + C1 * X_sq
22
23     # 2. Calculate Errors/Residuals
24     error = Y - Y_pred
25
26     # 3. Calculate Loss (MSE)
27     loss = np.mean(error**2)
```

```

28     history_loss.append(loss)
29     history_C0.append(C0)
30     history_C1.append(C1)
31
32     # 4. Calculate Gradients
33     grad_C0 = -(2/n) * np.sum(error)
34     grad_C1 = -(2/n) * np.sum(error * X_sq)
35
36     # 5. Update Parameters
37     C0 = C0 - learning_rate * grad_C0
38     C1 = C1 - learning_rate * grad_C1
39
40     # Print progress every 500 iterations
41     if i % 500 == 0 or i == n_iterations - 1:
42         print(f"Iteration {i:4d} | Loss: {loss:.5f} | C0: {C0:.5f}
43             | C1: {C1:.5f}")
44
45     print("-" * 50)
46     print(f"Training complete over {n_iterations} iterations")
47     print(f"Optimized Parameters: C0 = {C0:.4f}, C1 = {C1:.4f}")
48     print(f"Final MSE Loss: {history_loss[-1]:.6f}")

```

Out[3]:

```

Iteration    0 | Loss: 1.30516 | C0: 0.02239 | C1: 0.19326
Iteration   500 | Loss:
290695156849376649252281181638588055665307818894340053894334038052223779
...
Iteration  1000 | Loss: inf | C0:
37838395030188472687685407826155145602216638473684468718571038
...
Iteration  1500 | Loss: nan | C0: nan | C1: nan
Iteration  2000 | Loss: nan | C0: nan | C1: nan
Iteration  2500 | Loss: nan | C0: nan | C1: nan
Iteration  3000 | Loss: nan | C0: nan | C1: nan
Iteration  3500 | Loss: nan | C0: nan | C1: nan
Iteration  4000 | Loss: nan | C0: nan | C1: nan
Iteration  4500 | Loss: nan | C0: nan | C1: nan
Iteration  4999 | Loss: nan | C0: nan | C1: nan

-----

Training complete over 5000 iterations
Optimized Parameters: C0 = nan, C1 = nan
Final MSE Loss: nan

```

```

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\numpy
_core\_methods.py:132: RuntimeWarning: overflow encountered in ...
ret = umr_sum(arr, axis, dtype, out, keepdims, where=where)
C:\Users\Admin\AppData\Local\Temp\ipykernel_18124\3501735774.py:27:
RuntimeWarning: overflow encountered in ...
loss = np.mean(error**2)
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\numpy
_core\fromnumeric.py:83: RuntimeWarning: invalid value encountered in ...
return ufunc.reduce(obj, axis, dtype, out, **passkwargs)
C:\Users\Admin\AppData\Local\Temp\ipykernel_18124\3501735774.py:38:
RuntimeWarning: invalid value encountered in ...
C1 = C1 - learning_rate * grad_C1

```

Task 5 — ผลลัพธ์และการแสดงผล (Results & Visualizations)

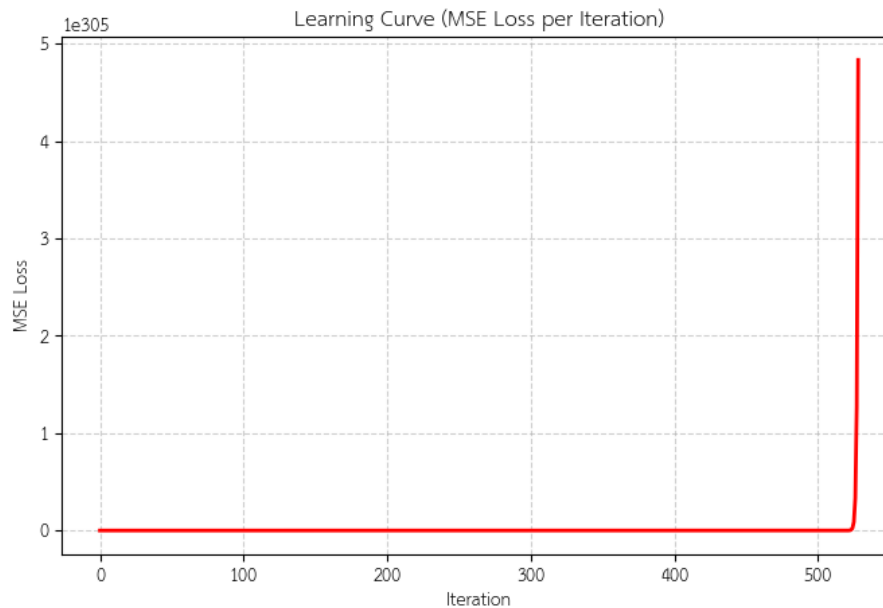
เราจะมาดูกันว่าโมเดลที่เราเทรนมามีพฤติกรรมอย่างไร และพิตกับข้อมูลได้ดีแค่ไหน

In [4]:

```

1 # กราฟที่ 1: การลดลงของ Loss (Learning Curve)
2 plt.figure(figsize=(8, 5))
3 plt.plot(range(n_iterations), history_loss, color='red', linewidth
    =2)
4 plt.title('Learning Curve (MSE Loss per Iteration)')
5 plt.xlabel('Iteration')
6 plt.ylabel('MSE Loss')
7 plt.grid(True, linestyle='--', alpha=0.6)
8 plt.show()

```

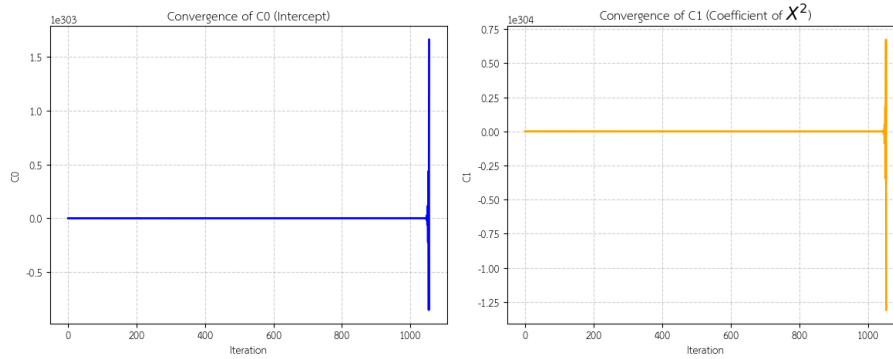



In [5]:

```

1 # กราฟที่ 2: เส้นทางการลู่อเข้าของพารามิเตอร์ C0 และ C1
2 plt.figure(figsize=(12, 5))
3
4 plt.subplot(1, 2, 1)
5 plt.plot(range(n_iterations), history_C0, color='blue', linewidth
6         =2)
7 plt.title('Convergence of C0 (Intercept)')
8 plt.xlabel('Iteration')
9 plt.ylabel('C0')
10 plt.grid(True, linestyle='--', alpha=0.6)
11
12 plt.subplot(1, 2, 2)
13 plt.plot(range(n_iterations), history_C1, color='orange',
14         linewidth=2)
15 plt.title('Convergence of C1 (Coefficient of  $X^2$ )')
16 plt.xlabel('Iteration')
17 plt.ylabel('C1')
18 plt.grid(True, linestyle='--', alpha=0.6)
19
20 plt.tight_layout()
21 plt.show()

```



In [6]:

```

1 # กราฟที่ 3: เส้นโค้งที่ฟิตแล้วเทียบกับจุดข้อมูลจริง (Fitted Curve)
2 plt.figure(figsize=(9, 6))
3
4 # จุดข้อมูลจริง
5 plt.scatter(X, Y, color='black', alpha=0.5, label='Actual Data',
6             edgecolor='k')
7
8 # สร้างจุด x สำหรับวาดเส้นโค้งให้เรียบเนียน
9 x_line = np.linspace(min(X), max(X), 100)
10 y_line = C0 + C1 * (x_line**2)
11
12 plt.plot(x_line, y_line, color='red', linewidth=3, label=f'Model:
13           $\hat{y} = {C0:.4f} + {C1:.4f}x^2$')
14
15 plt.title('Quadratic Model Fit')
16 plt.xlabel('X')
17 plt.ylabel('Y')
18 plt.legend(fontsize=12)
19 plt.grid(True, linestyle='--', alpha=0.6)
20 plt.show()

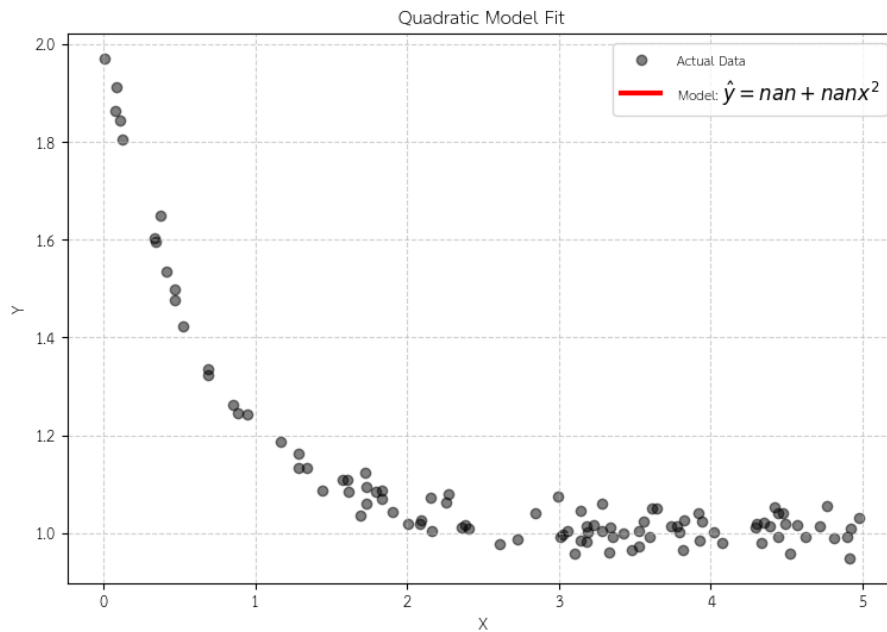
```

Out[6]:

```

<>:11: SyntaxWarning: invalid escape sequence '\h'
<>:11: SyntaxWarning: invalid escape sequence '\h'
C:\Users\Admin\AppData\Local\Temp\ipykernel_18124\1313811191.py:11:
  SyntaxWarning: invalid esca ...
  plt.plot(x_line, y_line, color='red', linewidth=3, label=f'Model:
    $\hat{y} = {C0:.4f} + {C1} ...

```



In [7]:

```

1 # กราฟที่ 4: ความคลาดเคลื่อน (Residuals) แต่ละจุด
2 Y_pred_final = C0 + C1 * X_sq
3 residuals = Y - Y_pred_final
4
5 plt.figure(figsize=(9, 5))
6 plt.scatter(X, residuals, color='purple', alpha=0.7, edgecolor='k'
7             )
8 plt.axhline(0, color='red', linestyle='--', linewidth=2)
9 plt.title('Residuals of the Model')
10 plt.xlabel('X')
11 plt.ylabel('Residual ($y_i - \hat{y}_i$)')
12 plt.grid(True, linestyle='--', alpha=0.6)
13 plt.show()

```

Out[7]:

```

<>:10: SyntaxWarning: invalid escape sequence '\h'
<>:10: SyntaxWarning: invalid escape sequence '\h'
C:\Users\Admin\AppData\Local\Temp\ipykernel_18124\1564452236.py:10:
  SyntaxWarning: invalid esca ...
  plt.ylabel('Residual ($y_i - \hat{y}_i$)')

```

